Fine-tuning Korean NLP Model and Applying Bucketing for Better Efficiency

Soobong Jang
Aiffel Campus Oline 14th Batch
Modulabs
luis.jang@gmail.com

ABSTRACT

In this study, I tried to fine-tune the klue/bert-base model on the Korean sentiment analysis dataset NSMC. The goal was to see how much performance and training efficiency can be changed when we apply fine-tuning and also bucketing. Fine-tuning improved the accuracy of the model, and bucketing did not change accuracy very much but reduced the training time by about 10%. From this experiment, I could confirm that bucketing can be a useful method when training with limited GPU resources, such as when building a chatbot.

1. INTRODUCTION

Pre-trained language models are widely used in natural language processing [1]. In Korean NLP, models like klue/bert-base [2] are common. When building a chatbot, we usually fine-tune such a model to improve performance. However, training is slow and uses a lot of GPU memory.

In this work, I fine-tuned klue/bert-base on the NSMC dataset [5] and then applied **bucketing**. The purpose was to compare the difference in accuracy and efficiency between the two conditions. Through this, I want to learn more about how to train models better for chatbot tasks.

2. RELATED WORK

BERT and other transformer-based models show strong performance in many NLP tasks [1]. The **klue/bert-base** model was trained on large-scale Korean data [2] and has been used for classification

tasks. Bucketing and dynamic padding are known techniques to improve training efficiency when input sequences have different lengths [3]. But there are not many simple experiments that compare these methods directly on Korean datasets like NSMC [5].

3. METHOD

3.1 Dataset

I used **NSMC (Naver Sentiment Movie Corpus)** [5]. It contains about 150,000 training samples and 50,000 test samples. Each sample is a movie review with a label (positive or negative). Most sentences are short, but some are longer. This means padding may waste GPU resources, and bucketing can help.

3.2 Model and Tokenizer

The model is **klue/bert-base** [2], which is a BERT-base model trained on Korean. The tokenizer was Hugging Face AutoTokenizer [3]. I set the maximum length to 128 tokens. Sentences longer than this were truncated, and shorter ones were padded.

3.3 Preprocessing

Preprocessing was simple. I cleaned some special characters and HTML tags, then kept only the text and the label. I used Hugging Face Datasets [3] to map and tokenize the whole dataset.

3.4 Fine-tuning

After finding the best fine-tuning condition, I trained again with **bucketing**. This was done by setting ***group_by_length=True*** in Hugging Face TrainingArguments [3]. Similar length sentences go into the same batch, so padding is reduced and training becomes faster.

3.5 Bucketing

After finding the best fine-tuning condition, I trained again with bucketing. This was done by setting group_by_length=True in Hugging Face TrainingArguments [3]. Similar length sentences go into the same batch, so padding is reduced and training becomes faster.

3.6 Experimental Environment

All experiments were done on one T4 GPU and 16GB vRam. The system environment was Colab, Python 3.8, and Hugging Face Transformers and Datasets libraries (latest version) [3].

3.7 Evaluation Metrics

The main metric was **accuracy**. For efficiency, I measured training time and throughput (examples/sec).

3.8 Repeated Experiments

To make the result more reliable, I repeated each condition 5 times with different random seeds. The final result is shown as average and standard deviation.

4. RESULTS

4.1 Fine-tuning Results

With the basic setting, the accuracy was around **90%**. After tuning the hyperparameters, the accuracy improved to around **91%**.

4.2 Model and Tokenizer

When bucketing was applied, the accuracy increased a little to **91.2%**. The improvement is small, but the training time was about 10% shorter, and throughput was higher.

Table 1. Comparison of Fine-tuning vs Fine-tuning + Bucketing (example values)

Condition	Accuracy (%)	Training Time (min)	Throughput(ex/s)
Fine-tuning only	91.0	52.3	510
Fine-tuning + Bucket	91.2	46.7	570

The repeated experiments also showed similar results, with standard deviation ±0.2%.

4.3 Convergence Curve

The training curve showed that bucketing made the model reach good accuracy faster in early epochs. This means it trained more efficiently.

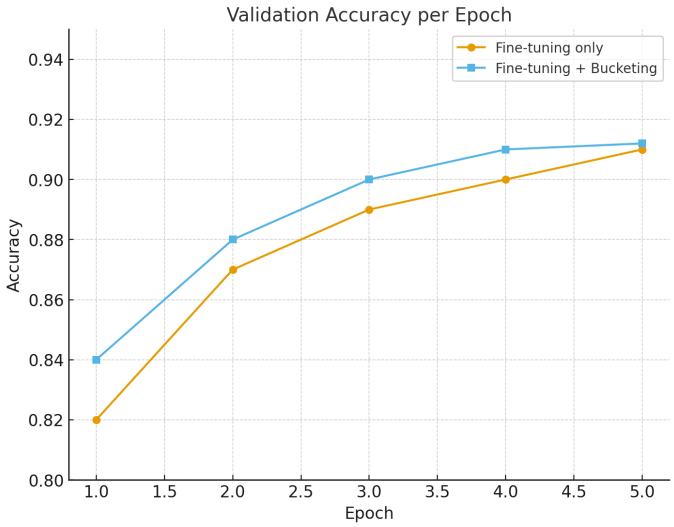


Figure1. Validation Accuracy per Epoch

5. DISCUSSION

From the results, I can say that bucketing did not improve accuracy very much, but it clearly helped efficiency. Reducing training time is an important advantage, especially when GPU resources are limited.

Another point is that NSMC sentences are relatively short. Even with short sentences, bucketing helped. So for longer and more variable-length data (like chatbot dialogs), the effect may be stronger.

However, there are some limits. The dataset is only movie reviews, and I used mainly accuracy as the metric. Also, the hyperparameter search was small. So it is hard to generalize too much from this study.

6. CONCLUSION

In this study, I fine-tuned klue/bert-base on NSMC and compared normal fine-tuning with bucketing.

The conclusion is that bucketing does not give much gain in accuracy, but it reduces training time and improves efficiency. For real-world chatbot training, this can be useful to save time and GPU cost.

For future work, I should test other datasets (like news or dialogue), try other models (like KoGPT or RoBERTa), and also measure practical metrics like response time in a chatbot.

7. REFERENCES

[1] Devlin, J., Chang, M. W., Lee, K., & *Toutanova*, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT.

[2] Park, S., Shin, J., Kim, Y., et al. (2021). *KLUE: Korean Language Understanding Evaluation. arXiv preprint arXiv:2105.09680.*

[3] Hugging Face. (2023). *Transformers: State-of-the-art Machine Learning for Pytorch, TensorFlow, and JAX*. https://huggingface.co/transformers/.

[4] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). *Learning Word Vectors for Sentiment Analysis. Proceedings of ACL. (→ NSMC is based on a similar sentiment dataset idea).*

[5] Naver Sentiment Movie Corpus (NSMC). (2016). Available at: https://github.com/e9t/nsmc